Vulnerability Assessment Report for a Live Website

Fatima Ilyas

10/06/2026

Table of Contents

Non-Disclosure Agreement

All information obtained during security assessment about Google Gruyere customer's systems and assets including, but not limited to, its procedures and systems, is deemed privileged information and not for public dissemination. Google Gruyere pledge their commitment that this information will remain strictly confidential. This information will not be disclosed or discussed to any third party without the express written consent of the customer. Google Gruyere is fully committed to maintaining the highest level of ethical standards in its business practice.

Legal Notice

It is impossible to test for 100% security. This report does not constitute, and should not be construed as a guarantee of the target's security.

Abstract

This report presents the results of a vulnerability assessment performed on Google Gruyere, a deliberately insecure web application designed to simulate real-world weaknesses, and explicitly hosted by security professionals to be legally attacked. The web application was subjected to multiple security evaluation techniques, including network scanning, automated vulnerability scanning, and human inspection of client-side elements. Tools like Nmap, OWASP ZAP, and Browser DevTools, were utilized to identify vulnerabilities that may be openly visible, or hidden, which were then analyzed, and given risk & confidence ratings. Finally, each vulnerability was given its respective mitigation steps, which could prevent future vulnerabilities, and strengthen the overall security posture of the application.

Introduction

Google Gruyere is a web-based application, designed to simulate real-world weaknesses, and is explicitly hosted by security professionals to be legally attacked. It provides core functionalities, typically seen in small-scale web applications, including user authentication, content display, and basic user interaction features.

The application interface presents structured content alongside user account management options such as registration and login, reflecting standard client-server interactions. Despite its simplicity, the underlying architecture, and implementation of the webpage could expose multiple potential security flaws that may arise from insufficient input validation, improper data handling, and outdated design practices.

This makes the application a suitable candidate for vulnerability assessment, as it represents common patterns observed in early-stage, or poorly secured web applications.

Scope

This assessment was conducted using a combination of automated, and manual testing techniques. Tools such as OWASP ZAP, were used to identify common web vulnerabilities, while Nmap was utilized, to analyze the network exposure of the hosting server.

Identified Vulnerabilities

OWASP ZAP was used for the identification of vulnerabilities, and was able to capture 9 alerts that compromised the safety of the web application. The risk severity of the captured vulnerabilities ranges from high, to medium, and low, and also includes information alerts, which refer to system findings, that leak data, but pose no direct, or actionable threats, or exploits on their own. Listed below are the vulnerabilities discovered, their risk, and confidence levels, and mitigation steps, on how to fix, and prevent these exploits from occurring in the future.

I. Cross Site Scripting (Reflected)

Cross Site Scripting, or XSS, refers to the practice where a malicious actor manipulates a vulnerable website by injected it with malicious code. When this code executes inside the user's browser when they visit the infected website, the attacker is able to fully compromise and interact with the user's device, and can gain access to the user's information. Reflected XSS attacks, also known as non-persistent attacks, are performed when an attacker embeds malicious script into a website's link. The script is activated, and the malicious code is run for the user when they click on the website's link, either provided through an email, comments sections, or social media posts [1].

Risk Level: High

Confidence Level: Medium

XSS attacks can be prevented mainly by user vigilance, and training, and informing individuals not to click on suspicious links from emails, comments sections, or unknown users. This allows users to gain awareness, and provide them with the knowledge to not trust every link, or user on the internet. Another method is for a web application firewall, or WAF, to be implemented by a website's host, to prevent, and block user attacks by blocking abnormal requests [2].

II. Content Security Policy (CSP) Header Not Set

Content Security Policy (CSP), is the name of an HTTP response header that modern browsers use to enhance client-end security of a web page. It allows the developer of the website to restrict, and filter what resources, such as JavaScript scripts, CSS stylesheets, and media files, can be loaded and displayed to a user, and the URLs they can be loaded from. This header reduces the likeliness of attacks such as XSS, and data injection attacks from occurring, which explains why this webpage was a target of an XSS attack [3].

Risk Level: Medium

Confidence: High

To prevent this error, the host of the web page must configure their web server or application to send a CSP header, with restrictive directives. The host must start with a minimal policy to enable enforcement, then tune it to match their application's needs [3].

III. Missing Anti-clickjacking Header

Clickjacking is when an attacker layers multiple transparent, or opaque elements over legitimate elements on a web page, which causes users to click on the malicious elements, rather than the elements they originally intended to click on. This causes the user to be re-routed to another page, application, or domain, most likely owned by the attacker. The anti-clickjacking header, primarily known as X-Frame-Options (XFO), tells a browser whether the contents of a webpage should be loaded into specific elements (like <iframe>, <frame>, or <object>) by another webpage, which allows for the prevention of users clicking invisible elements placed by attackers.

Risk Level: Medium

Confidence: Medium

To prevent this attack from occurring, web application hosts should consider implementing the XFO header onto their website, and ensure it is enabled on all webpages that return back to their website. This is to

ensure no single element from the host webpage is able to fall vulnerable to a clickjacking attack leading to an attacker's site, or application [4].

IV. Cookie Not Containing HttpOnly Flag

The HttpOnly flag is an extra addition that can be added to a response header (Set-Cookie). This flag allows browsers to keep cookies out of JavaScript APIs, which are ready-made code blocks that perform simple tasks for a webpage without having to code them from scratch. When this flag is enabled, running APIs cannot read the values of cookies, which enhances the safety, and privacy of users. This flag also prevents common attacks like XSS attacks, where attackers want to gain access to user cookies. However, when this flag is not enabled, attackers can easy gain access to active session cookies through running JavaScript scripts, which increases the vulnerability of the webpage.

Risk Level: Low

Confidence: Medium

Web application hosts must ensure the HttpOnly flag is set for all cookies, across all required pages of their site. The flag works by an empty string returning by the browser when an accepted cookie is detected by the HttpOnly flag, and the client-side script code attempts to read it [5].

V. Cookie Without Secure Flag

The secure flag is an attribute of a cookie that instructs a browser to only send a cookie over an encrypted HTTPS connection. When the secure flag is not implemented, cookies can be accessed by attackers if a user accesses a non-encrypted HTTP website, and can allow them to gain sensitive, private user information like user preferences, login details, and tracked user activity.

Risk Level: Low

Confidence: Medium

To prevent this vulnerability from occurring, the webpage host is encouraged to ensure the secure flag is enabled across all pages of their website/application, to ensure cookies that contain sensitive information, are sent over an encrypted channel [6].

VI. Cookie Without SameSite Attribute

The SameSite attribute prevents browsers from sending essential cookies along cross-site requests, to mitigate critical risks, such as protection against cross-side request forgery (CSRF) attacks. CSRF attacks

take place when a user accesses a website with accepted cookies, along with accessing a malicious website that contains hidden scripts that trigger cross-site POST requests targeting the intended website. The user's browser responds to the cross-site request, which automatically attaches the user's cookie to the attacker's website.

Risk Level: Low

Confidence: Medium

According to OWASP ZAP, website hosts must enable the SameSite flag at either lax, or strict values. The strict value prevents cookies from being sent from a browser to targeted sites in all possible cross-site scenarios, even if the targeted site is not malicious. The lax value, however, provides a balance, and allows cookies to be sent only during same-site requests, and cross-site requests, only if they are considered safe [7].

VII. HTTPS Content Available via HTTP (Systemic)

The following vulnerabilities are systemic, meaning they are widespread all across the website, and the pages related to it, and are not just present as isolated glitches. This vulnerability allows content that would generally only be accessible via HTTPS to also be accessible via HTTP. Without a secure connection, any data transmitted over HTTP can efficiently be accessed by attackers via man-in-the-middle, or data interception attacks. This attack also causes trust, reputation, and legal violations, as user trust is lost due to inefficient security, which causes the webpage to be avoided, effecting its popularity and usage. Legal issues include failure to enforce HTTPS, which can result in compliance violations [8].

Risk Level: Low

Confidence: Medium

To mitigate this vulnerability, webpage hosts are encouraged to ensure all servers including web, and application servers, load balancers, etc., are configured only to serve the content they provide via an encrypted HTTPS connection. OWASP ZAP recommends implementing the HTTPS Strict Transport Security header, which informs browsers that the domain needing to be accessed should only be accessed using HTTPS [9]. Webpage hosts can also configure their web servers to redirect any HTTP requests, to their corresponding HTTPS counterparts, and implementing a Content Security Policy, which restricts the loading of content from insecure sources, preventing the mixing of content, and ensuring all resources are loaded securely via HTTPS [8].

VIII. Strict-Transport-Security Header Not Set (Systemic)

As mentioned previously, the Strict-Transport-Security Header is vital, considering it informs browsers that

any and all domains accessed by a particular user, should only be accessed via HTTPS, and notifies the browser that any future attempts to access that domain via HTTP should automatically be upgraded to HTTPS. Additionally, when connecting to the domain in the future, the browser will not allow the user to bypass secure connection errors, such as invalid certificates (when a browser is unable to verify a website's security due to an expired or untrusted authority of a certificate) [10].

Risk Level: Low

Confidence: High

The webpage host should consider enabling the Strict-Transport-Security header across all pages of their website.

IX. X-Content-Type-Options Header Missing (Systemic)

As detected by OWASP ZAP, he Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff', which allows older versions of browsers to perform MIME-sniffing on the client-side, potentially causing the client-side to be interpreted and displayed as a content type other than the declared content type. To explain, MIME types, currently known as media types, indicate the nature, and format of a document, file, or assortment of bytes [11]. MIME-sniffing is the practice, adopted by browsers, to determine the effective MIME, or media type of a piece of content on a web application, by examining the content of the response, instead of relying on the Content-Type header [12]. The act of MIME-sniffing may seem harmless, but it can be a serious vulnerability, as it can exploit browsers' attempts to guess file types, which turns harmless-looking files into serious security risks [13].

Risk Level: Low

Confidence: Medium

Web authors are encouraged to enable the X-Content-Type-Options header, and set it with the value "nosniff", which tells browsers to strictly follow the declared content-type, rather than sniffing, and determining the file type itself. Another practice involves always validating uploaded files on a server, along with ensuring that the file's actual content matches its declared MIME type [13].

Informational Vulnerabilities

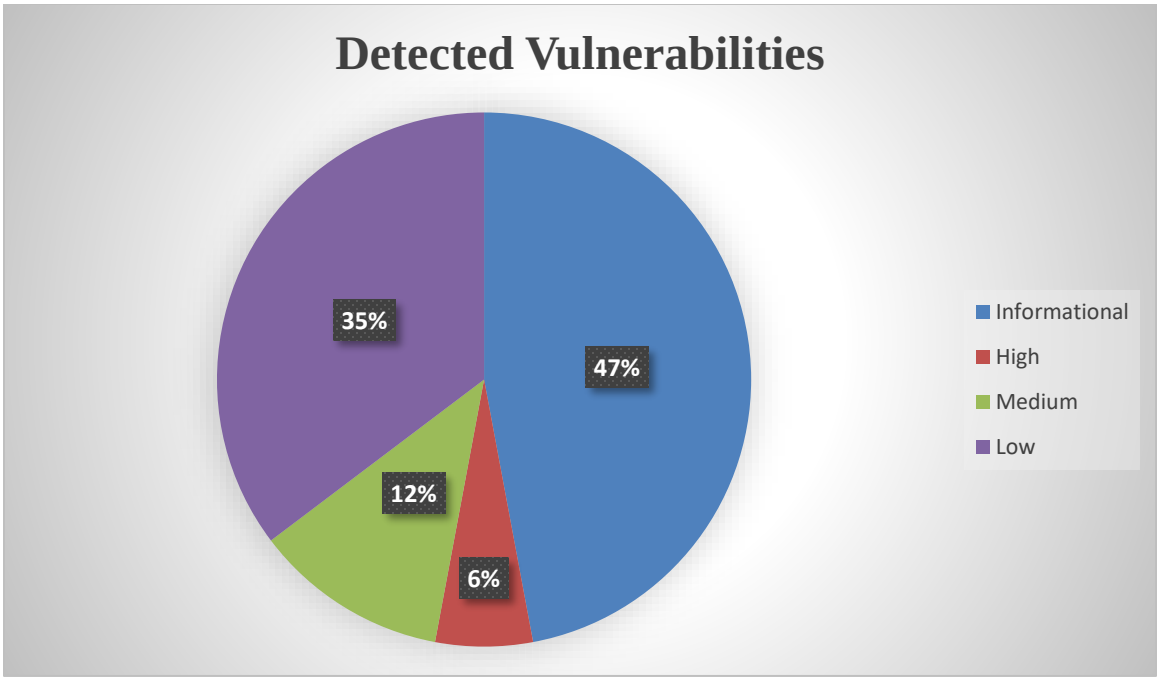
When conducting a vulnerability scan, a scanner may detect useful information that does not necessarily pose any real risk, such as installed software, open ports, or general information about a system, and how it operates. This information is classified as informational vulnerabilities, or otherwise, potential vulnerabilities [14]. Some informational vulnerabilities detected in the scan for Google Gruyere include, but are not limited to:

1. **Charset Mismatch (Header Versus Meta Content-Type Charset) (Systemic):** occurs when two components of a software ecosystem, such as the HTML file, and the HTTP Content-Type header of a website, try to read, write, or compare, the text data using different character encodings (for example, comparing UTF-8 with ISO-8859-1), which can trigger security warnings, or break special characters [15]. Forcing a single character encoding for all text content in both headers, and files, fixes this issue.
2. **Cookie Poisoning:** an attack strategy in which an attacker alters, forges, or injects, an otherwise valid cookie with malicious content as it is sent back to a server after user confirmation, to steal data, or bypass security [16]. The aim of cookie poisoning is to change the content of a cookie before it is received by the web application, otherwise known as cookie manipulation [17]. Occurs during attacks like XSS (cross-site scripting), packet sniffing (interfering with data packets travelling across a computer network), and brute force (cracking cookies through continuous, and rigorous trial-and-error). When designing a web application, hosts must ensure the content exchanged through cookies cannot be easily manipulated by the sender, enforce HTTPS for all pages, and promote the use of VPNs, especially for insecure connections.
3. **Information Disclosure - Suspicious Comments:** suspicious comments in the production code of, or in web applications themselves, can reveal sensitive information, such as development notes, system details, passwords, debugging information, or business logic, which could assist attackers. Removing development comments, and reviewing comment content before deployment, and sanitizing error messages to ensure they do not contain sensitive information, can remediate this vulnerability [18].
4. **Modern Web Application (Systemic):** an informational alert, does not require changes, or remediations.
5. **Re-examine Cache-control Directives (Systemic):** cache-control header (holds instructions in both requests from the client-side, and responses from the server-side, that control cached information in browsers, and shared caches) has been set improperly, or is missing. Ensure the cache control HTTP header is set with the values "no-cache, no-store, must-revalidate", which instructs browsers, and intermediate caches, to never cache a specific webpage, or asset, never store, or write a response to disk or memory, and ensures users always fetch the latest, fresh copy directly from the origin server [19].
6. **Retrieved from Cache (Systemic):** content retrieved is from a shared cache, which, if the response data is sensitive, personal, or user-specific, like financial details, can cause the information to be leaked. This may also result in a user gaining complete control of a session of another user. This occurs when caching servers are configured on local networks, for instance, in corporate, or educational environments [20]. Validating that responses do not contain sensitive, personal, or user-specific information, and considering the use of relevant HTTP response heads, can limit, or prevent this vulnerability from arising.
7. **Session Management Response Identified:** an informational alert, does not require changes, or remediations.

8. **User Agent Fuzzer (Systemic):** a testing technique used to identify vulnerabilities in web applications, works by manipulating the user-agent header (request header that sends a characteristic string to web servers, allowing them to identify the operating system (OS), and browser of the client making the request [21]), by injecting unexpected strings into a client-side request, to examine how the server-side responds to the processed information.

Graphical Overview

Displayed below, is a graphical representation of the risk levels of all the vulnerabilities detected during the OWASP ZAP scan performed on Google Gruyere.



Conclusion

This vulnerability assessment conducted on Google Gruyere's web application revealed multiple security weaknesses, across both client-side, and server-side components. The use of tools such as Nmap, for network information, such as the detection of the application's ports, and their statuses, information about operating system the application is being hosted from, and detection of vulnerabilities using the "- - scripts vuln" command, and OWASP ZAP, for detailed information about the vulnerabilities detected on the application, combined with manual inspection techniques such as inspection using Google's built-in inspect element for header information, enabled the identification of a range of vulnerabilities, including high-risk

issues, like Cross-Site Scripting, as well as several medium, and low-risk weaknesses related to missing security headers, and improper cookie handling.

These findings highlight the impact of insufficient security controls, particularly in areas such as browser-side protection, and input validation. The recommended mitigation techniques, including the implementation of secure HTTP headers, enforcement of HTTPS, and improved cookie management, are essential steps towards strengthening the security posture of the web application.

Overall, this assessment demonstrated the importance of proactive security testing, and secure development practices, in identifying and addressing vulnerabilities, before they can be exploited in real-world environments.

References

1. "What Is Cross-Site Scripting (XSS) and How to Prevent It? | Web Security Academy." Portswigger.net, portswigger.net/web-security/cross-site-scripting#what-is-cross-site-scripting-xss.
2. Imperva. "Reflected XSS | How to Prevent a Non-Persistent Attack | Imperva." Learning Center, 2019, www.imperva.com/learn/application-security/reflected-xss-attacks/.
3. "Content Security Policy (CSP) Header Not Set: Vulnerability & How to Fix It." Invicti.com, 2026, www.invicti.com/blog/web-security/content-security-policy-csp-header-not-set-vulnerability-how-to-fix-it.
4. Rydstedt, Gustav. "Clickjacking | OWASP." Owasp.org, 2022, owasp.org/www-community/attacks/Clickjacking.
5. "X-Frame-Options - HTTP | MDN." MDN Web Docs, 13 Mar. 2025, developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options.
6. OWASP. "HttpOnly - Set-Cookie HTTP Response Header | OWASP." Owasp.org, owasp.org/www-community/HttpOnly.
7. "HawkScan Test Info for Cookie without Secure Flag." Stackhawk.com, 12 Sept. 2024, docs.stackhawk.com/vulnerabilities/10011/.
8. "SameSite | OWASP Foundation." Owasp.org, owasp.org/www-community/SameSite.
9. "HTTPS Content Available via HTTP | StackHawk Documentation." StackHawk Documentation, 2019, docs.stackhawk.com/vulnerabilities/10047/.
10. "Strict-Transport-Security - HTTP | MDN." MDN Web Docs, 13 Mar. 2025, developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security.
11. "X-Content-Type-Options - HTTP | MDN." MDN Web Docs, 13 Mar. 2025, developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options.
12. K, Natarajan C. "Understanding MIME Sniffing Attacks: A Real-Life Example." Medium, 13 May 2025, medium.com/@natarajanck2/understanding-mime-sniffing-attacks-a-real-life-example-7151e1a5d9a5.

13. “ZAP – Charset Mismatch (Header versus Meta Content-Type Charset).” Zaproxy.org, 2017, www.zaproxy.org/docs/alerts/90011-1/.
14. “What Is Cookie Poisoning?” Www.f5.com, www.f5.com/glossary/cookie-poisoning.
15. “Cookie Poisoning.” Invicti.com, 2026, www.invicti.com/learn/cookie-poisoning.
16. “HawkScan Test Info for Information Disclosure - Suspicious Comments.” Stackhawk.com, 19 Sept. 2025, docs.stackhawk.com/vulnerabilities/10027/.
17. “OWASP ZAP – Re-Examine Cache-Control Directives.” Www.zaproxy.org, www.zaproxy.org/docs/alerts/10015/.
18. “Cache-Control - HTTP | MDN.” MDN Web Docs, 13 Mar. 2025, developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control.
19. “ZAP – Retrieved from Cache.” Zaproxy.org, 2017, www.zaproxy.org/docs/alerts/10050-2/.
20. GeeksforGeeks. “HTTP Headers UserAgent.” GeeksforGeeks, 11 Oct. 2019, www.geeksforgeeks.org/html/http-headers-user-agent/.